

Paper Title	Method	Dataset	Advantages	Limitations	Research Gap (Verified)
laC Generation with LLMs: An Error Taxonomy and A Study on Configuration Knowledge Injection	Evaluates LLM-based Terraform code generation across Naive RAG, Graph RAG, and three graph enhancements (GR-OptMatch, GR-LLMSum, GR-Ref) using cloud emulation (LocalStack) and automated validation.	laC-Eval benchmark enhanced with LocalStack, comprising 458 Terraform generation prompts and official AWS Provider v5.90.0 documentation.	Boosts technical validation success from 37.2% to 84.2% and overall success from 27.1% to 62.6%. Fully local execution via LocalStack eliminates AWS account costs, network latency, and accidental resource provisioning. Provides a detailed two-dimensional taxonomy of 19 error types.	Intent validation plateaus at 72%-76% due to the 'Correctness-Congruence Gap'. Graph RAG with full reference modeling (GR-Ref) causes cognitive overload and degrades performance on simpler prompts. Restricted exclusively to Terraform configurations for AWS.	Existing knowledge-injection approaches focus narrowly on single-provider (AWS) Terraform syntax, leaving an intent-alignment gap (the Correctness-Congruence Gap) and no mechanism to optimize context retrieval across the heterogeneous, multi-format laC stack this project targets.
GENSIAC: Toward Security-Aware Infrastructure-as-Code Generation with Large Language Models	Instruction fine-tuning of base LLMs (CodeLlama 7B/13B) using LoRA (Low-Rank Adaptation) and 8-bit quantization to inject security-weakness awareness into generation and inspection workflows.	GENSIAC Dataset: 22K data points (11K code generation, 11K code inspection pairs covering Ansible, Chef, Puppet across 1,413 repos) + 148 test scripts (229 security flaws).	Raises security-weakness detection F1-score from ~0.30 to 0.858 (outperforming GPT-4's 0.592). Improves syntactical and functional correctness scores to 1.000 and 0.979. Resource-efficient fine-tuning requiring under 10 hours per epoch on an A100 GPU.	Does not cover all security weakness categories (e.g., CWE-276) due to reliance on upstream static-detection tools. Cross-language transfer is weaker on Ansible than on Chef/Puppet. Only tested with LoRA; no runtime cloud validation.	Security-aware fine-tuning here still depends on static scanners for training labels and stops at code inspection, with no live deployment verification — a gap this project's security-assessment agent closes by pairing static checks with runtime telemetry.
INFRAFIX: Technology-Agnostic Repair of Infrastructure as Code	A polyglot Automated Program Repair (APR) framework using an extended object-oriented Intermediate Representation (IR), a normalization database, system-call-based state inference, and an SMT-based repair engine.	Curated Polyglot Dataset: 12,358 Ansible, 4,868 Puppet, 3,568 Chef, 1,300 Terraform, and 13 Tortoise scripts, yielding 254,288 repair scenarios.	Technology-agnostic architecture operating across Ansible, Puppet, Chef, and Terraform simultaneously. High overall repair success rate of 95.7% across 254,288 scenarios. Includes an automated mutation-based repair-scenario generator.	Fails when resource names cannot be statically resolved into strings or are defined in uninstantiated classes/functions. Struggles with undefined-variable references and shared-variable side effects. Requires separate state-inference modules for non-filesystem resources.	Symbolic, constraint-driven repair cannot dynamically resolve runtime variables or cloud-resource dependencies without hand-written rules — motivating this project's use of LLM-agent reasoning alongside static repair for dynamically evolving smart-manufacturing infrastructure.
GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation	End-to-end framework pairing GenKubeDetect (CodeT5+ fine-tuned with LoRA for multi-label classification) with GenKubeResolve (Mistral-7B guided by few-shot prompts for localization, reasoning, and remediation).	Unified Misconfiguration Index (UMI) with 169 misconfig classes + 276,520 Kubernetes Configuration Files (KCFs) from 'The Stack' + 700MB domain text.	High detection precision (0.990) and recall (0.999) versus static rule-based tools. Provides 100% expert-validated line-level localization, reasoning, and remediation. Adapts to new misconfigurations from just 1-5 labeled examples.	Constrained by CodeT5+'s 512-token input limit, leaving 17% of longer KCFs unanalyzed. High false-positive rates during few-shot adaptation on similar categories (e.g., RBAC rules). Qualitative evaluation limited to 100 false positives and 30 resolutions.	Token-limited, Kubernetes-only detection models cannot scale to the multi-service AWS architectures (EC2, S3, IAM, Lambda) this project must validate, nor do they coordinate with agents handling other laC formats in one pipeline.
Practical Integration of Large Language Models into Enterprise CI/CD Pipelines for Security Policy Validation: An Industry-Focused Evaluation	Enterprise DevSecOps framework augmenting static scanners (OPA/Checkov) with an LLM Ensemble (GPT-4, domain fine-tuned models, and ReAct input-sanitization filtering) inside Jenkins/Bamboo CI/CD pipelines.	Synthetic laC Suite: 500 cases (300 K8s, 200 IAM/Terraform) + real-world validation on 318 enterprise Helm charts (~47k LOC).	Ensemble method achieves high classification accuracy (F1 = 0.95) at low latency (3.1s). ReAct input filtering cuts prompt-injection attack success from 60% down to 7%-12%. Reduces manual security-review time by 60% and monthly rule maintenance by 70%.	Ensemble approach requires 3x computational resources, raising operational cost. Generates false alarms (~23% hallucination rate) and is over-confident on ambiguous code. Evaluation restricted to mid-scale, largely synthetic workloads.	High-cost ensemble validation with a ~23% hallucination rate highlights the need for a cost-optimized, coordinated multi-agent pipeline that pairs automated validation with real-time cloud telemetry — exactly the gap this project's Lambda-orchestrated agent architecture is designed to fill.
SWE-InfraBench: Evaluating Language Models on Cloud Infrastructure Code	Introduces SWE-InfraBench, a benchmark for evaluating LLMs on incremental Infrastructure-as-Code (laC) editing tasks in AWS CDK repositories using natural-language instructions and automated unit tests.	100 instruction-based tasks collected from 34 real-world AWS CDK repositories.	Focuses on realistic enterprise workflows; supports imperative laC editing; uses automated validation against real-world repositories.	Restricted mainly to AWS CDK. Does not fully analyze deployment reliability, security compliance, or production-scale environments.	A single-format (CDK) editing benchmark cannot verify cross-format deployment reliability or security compliance — this project's framework must jointly validate CDK alongside Terraform, CloudFormation, Kubernetes, and SAM within one workflow.
Multi-laC-Eval: Benchmarking Cloud Infrastructure as Code Across Multiple Formats	Proposes Multi-laC-Bench, a benchmark for laC generation and mutation across CloudFormation, Terraform, and CDK using natural-language requests.	Synthetic triplets consisting of an initial template, a modification request, and an updated template generated from public repositories.	Covers multiple laC languages; supports both generation and mutation; introduces standardized cross-format evaluation metrics.	Relies heavily on synthetic data and semantic evaluation via LLM judges and static-analysis tools rather than live deployments.	Multi-format coverage here is evaluated only on synthetic templates with no deployment-time or security feedback — this project extends multi-format validation with live AWS deployment verification and runtime telemetry, which synthetic benchmarks cannot provide.
Deployability-Centric Infrastructure-as-Code Generation: Fail, Learn, Refine, and Succeed through LLM-Empowered DevOps Simulation	Introduces laCGen, an iterative framework that refines generated templates using format verification, syntax checking, and live deployment feedback.	DPIaC-Eval benchmark containing 153 real-world scenarios across 58 AWS services.	Moves beyond syntax by emphasizing deployability; mirrors DevOps workflows; incorporates iterative feedback and human guidance.	Requires repeated deployment attempts and human intervention, increasing cost and reducing scalability. User-intent alignment and security compliance remain weak.	Iterative, human-in-the-loop refinement does not scale automatically and leaves security compliance unaddressed — this project's multi-agent design must automate refinement while adding a dedicated security-assessment stage that laCGen lacks.

Paper Title	Method	Dataset	Advantages	Limitations	Research Gap (Verified)
RIVA: Leveraging LLM Agents for Reliable Configuration Drift Detection	Proposes RIVA, a multi-agent architecture with verifier and tool-generation agents that collaborate through cross-validation and tool-history tracking.	Evaluated on the AIOpsLab benchmark containing infrastructure localization and drift-detection tasks.	Improves robustness against faulty tools and enhances configuration-drift-detection accuracy.	Focuses mainly on configuration-drift detection and is evaluated only in benchmark settings, not real cloud deployments.	Drift detection is treated as an isolated, benchmark-only capability rather than one stage in a coordinated pipeline — this project integrates drift detection with generation, security assessment, and cost analysis using real CloudWatch/telemetry data rather than a static benchmark.
TerraProbe: A Layered-Oracle Framework for Detecting Deceptive Fixes in LLM-Assisted Terraform Security Repair	Introduces TerraProbe, a five-layer oracle framework for evaluating LLM-generated Terraform security repairs.	288 first-pass repairs generated across 68 real-world Terraform modules and 28 injected-defect modules.	Evaluates planning validity and security intent in addition to scanner results; includes statistical analysis and multi-model comparisons.	Limited to Terraform security repair and emphasizes evaluation rather than automatic secure-repair generation.	Rigorous evaluation of 'deceptive fixes' is confined to Terraform and stops short of generating the repair itself — this project needs equivalent deception-aware verification generalized across all supported IaC formats, feeding directly into automated remediation.
Large Language Models as Configuration Validators	Develops a generic LLM-based configuration validation pipeline and empirically evaluates multiple LLMs (e.g., GPT-4) on their ability to detect misconfigurations by prompting them with configuration values and system context.	Real-world configuration files and historical misconfiguration issues drawn from open-source systems (e.g., HDFS, HBase, Spark) used in prior machine-learning-based configuration validation research.	First systematic feasibility study proving LLMs can validate configurations without large-scale labeled field data or system-specific trained models, overcoming key limitations of ML-based validators.	Evaluated only on general-purpose software systems, not industrial or manufacturing control configurations; no retrieval grounding, so accuracy depends purely on the LLM's internal knowledge of each system.	Unclear whether this validation paradigm transfers to real-time, safety-critical manufacturing configurations (PLC/SCADA), where a missed misconfiguration causes physical, not just software, failure.
Agents4PLC: Automating Closed-Loop PLC Code Generation and Verification in Industrial Control Systems Using LLM-Based Agents	Multi-agent LLM framework combining Retrieval-Augmented Generation (RAG), advanced prompt engineering, and Chain-of-Thought reasoning to generate and formally verify PLC code end-to-end.	A purpose-built benchmark of natural-language PLC requirements paired with human-written, formally verified reference PLC code and specifications.	Demonstrates that RAG-enhanced multi-agent verification substantially outperforms single-shot LLM code generation for industrial control code, with measurable correctness gains.	Focused specifically on PLC/IEC-61131 code; does not extend to other manufacturing infrastructure layers such as SCADA networks, MES configuration, or edge orchestration.	No unified framework validates infrastructure code across the full manufacturing stack (PLC + SCADA + MES + edge) rather than treating PLC code generation in isolation.
LLM4PLC: Harnessing Large Language Models for Verifiable Programming of PLCs in Industrial Control Systems	A user-guided iterative pipeline combining LLM code generation with external verification tools (grammar checkers, compilers, SMV model checkers) and LoRA fine-tuning to progressively correct PLC program errors.	Validated on a FischerTechnik Manufacturing TestBed (MFTB), a physical smart-manufacturing testbed, alongside benchmark industrial control programming tasks.	Shows a clear, measurable evolution from structurally flawed to formally verifiable PLC code; real hardware validation on an actual manufacturing testbed strengthens practical relevance.	Still requires iterative human feedback in the loop, limiting full automation; depends on external formal verification tools that must be manually configured per PLC dialect.	No fully autonomous, continuously learning validation loop yet removes the need for repeated human feedback across diverse manufacturing hardware and PLC dialects.
Application of Large Language Models in the Protection of Industrial IoT Systems for Critical Infrastructure	Analytical study examining industrial automation system architecture and IIoT cyber-threat scenario classes, and assessing how LLMs could support monitoring, incident analysis, and compliance automation across IT/OT layers.	Not an empirical dataset-driven study; draws on architectural analysis of industrial automation systems and existing IIoT threat taxonomies and standards (e.g., ISO/IEC 62443, NIS2).	Provides a structured view of IT/OT convergence risks and positions LLMs as a tool for automating compliance and threat analysis specific to critical industrial infrastructure.	Conceptual and analytical rather than an implemented, empirically validated system; does not perform configuration or code-level validation directly.	Leaves an operational gap between "LLMs could help secure IIoT" and an actual, implemented, validated IaC/configuration framework for manufacturing environments.
From Words to Wires: Toward Rapid ICS Cyber-Range Construction Using LLMs	A lightweight LLM-integrated toolchain (Words2Wires) that converts plain-language descriptions into runnable ICS simulation artifacts — control scripts, simulation code, and OPC-UA tag maps — executed directly.	Example ICS scenarios (e.g., tank systems, traffic-light systems) used to demonstrate rapid testbed generation, with execution benchmarked at under 90 seconds per instance.	Demonstrates that LLMs can generate functioning, network-visible ICS infrastructure directly from natural language, dramatically cutting testbed setup time and vendor-specific tooling overhead.	Generated artefacts are executed "without manual edits or formal validation" — meaning the LLM-generated infrastructure code is never checked for correctness or security before running.	By the authors' own admission, no validation layer exists for LLM-generated industrial infrastructure code — precisely the gap your proposed framework is designed to fill.